Lecturas de una estrategia

AlgoProject — manual de uso. Actualizado el 2026-10-07 con `tools/manual.py`.

1. 26. Nulos — ¿esto lo habría conseguido un mono entrando al azar?
2. 50. Edge sobre el coste pagado
3. 42. La calidad de la entrada — ¿la señal vale algo por sí sola?
4. 53. El mapa condicional — ¿de qué depende que esto siga funcionando?
5. 51. Estructura — qué condición sostiene el filo, y si vive en la dirección de la entrada

26. Nulos — ¿esto lo habría conseguido un mono entrando al azar?

Qué pregunta responde

Coge las operaciones reales de una estrategia y fabrica miles de versiones imaginarias de ella sobre **las mismas velas**: el mismo número de operaciones, la misma duración, el mismo dinero en riesgo y el mismo coste de bróker — cambiando solo **cuándo entra**. Luego te dice en qué percentil de esas miles quedó la real.

Si el resultado de tu estrategia lo consigue una de cada tres tiradas al azar, no hace falta discutir más. Si no lo consigue ninguna de 2.500, has medido algo.

Y hace una segunda cosa más interesante: como puedes elegir **qué le dejas fijo al mono**, la diferencia entre dos configuraciones te dice de dónde sale el edge — del momento de entrar, de cuánto aguanta la posición, o de cómo calcula el tamaño.

Cuándo lo usas, y cuándo no

Lo usas después de exportar las operaciones de una databank, sobre el tramo `oos1`, para decidir qué estrategias merecen que el pipeline gaste días de máquina en ellas.

Cada lectura del tramo `oos2` queda apuntada en el ledger; mirarlo ya no está prohibido a un humano (dueño, 2026-09-28).

No sirve para decirte si funcionará el año que viene. Sí puede decirte, **mercado a mercado**, si la entrada bate al mono en cada uno de los mercados del retest cross-market (abajo), pero cada mercado se juzga contra sus propios monos: no pone dos mercados en la misma escala. Lo que mide es: *dada esta historia concreta, ¿hubo habilidad en el momento de entrar?* Si lo que quieres saber es qué pasaría en **otra** historia, eso es el MC Retest (página 11).

Antes de empezar

1. Tienen que existir las operaciones exportadas en `raw/<proyecto>/<databank>/<fecha>/trades.parquet` (página 06 para exportarlas).
2. Tienen que existir las velas del feed en `bars/` (página 13).
3. **No hace falta cerrar SQX.** Este módulo no toca la instalación: lee ficheros ya exportados.

Cómo se ejecuta

Una sola estrategia, que es el uso normal:

```
python3 -m studies.readings.monkey.one --project XAUUSD --databank Results \
    --feed XAUUSD_DukasM1_Infinox --strategy "Strategy 1.17.44"
```

Primero la verificación, siempre. Sobre una estrategia cualquiera:

```
python3 -m studies.readings.monkey.verify --project XAUUSD --databank Results \
    --feed XAUUSD_DukasM1_Infinox --strategy "Strategy 1.17.44"
```

Y luego la corrida completa:

```
python3 -m studies.readings.monkey.report --project XAUUSD --databank Results \
    --feed XAUUSD_DukasM1_Infinox --timeframe M30 --sample OOS1
```

flag	obligatorio	qué hace
`--project`	sí	nombre del proyecto tal y como aparece en SQX
`--databank`	sí	databank cuyo export de operaciones se lee; coge el más reciente
`--feed`	sí en un export de un mercado	nombre del feed en SQX, p.ej. `XAUUSD_DukasM1_Infinox`: qué velas usar. En un export cross-market, **sin `--feed` prueba todos los mercados**, cada uno con sus velas; con `--feed` solo ese
`--timeframe`	no	`M30` por defecto. Tiene que ser el de la estrategia o las duraciones no cuadran
`--sample`	no	`OOS1` por defecto. `IST` es dentro de muestra — úsalo para atribuir, nunca para decidir
`--limit`	no	solo las N primeras estrategias (en todos sus mercados), para una prueba rápida
`--set`	no	cambia un ajuste sin editar el fichero: `--set nulls.draws=500`
`--seed`	no	la semilla de los monos. **Sin él, cada corrida sortea una nueva** (decisión del dueño, 2026-09-29) y sus p cambian un poco de una vez a otra; con él, `--seed 20261002`, dos corridas dan los mismos p, byte a byte. La que se usó queda siempre apuntada: en la última línea que imprime, en `manifest.json` (`seed` , y dentro del comando para repetirlo) y en la columna `seed` de `nulls.csv`
`--workers`	no	pares estrategia-mercado a la vez, uno por proceso. Por defecto los 48 núcleos físicos, que es también el máximo

Mercado a mercado. Sobre el export del retest cross-market (el de la página 05), sin `--feed`:

```
python3 -m studies.readings.monkey.report --project USDJPY_workflow_profiling_v1 \
    --databank "Retest Markets - Family" --timeframe H1 --sample IST
```

`nulls.csv` sale con una fila por estrategia **y mercado** (columna `market`). El valor del punto y el coste se miden en cada mercado con sus propias operaciones: la reconciliación con SQX sale 0,9999 en los diez mercados del USDJPY. `monkeyExcess` da después un informe por mercado, en `monkeyExcess/<feed>/`. Un export de varios mercados leído estrategia a estrategia **sin decir cuál** se niega: antes mezclaba las operaciones de diez mercados como si fueran una.

Cuánto tarda: 757 estrategias × 4 peldaños × 2.500 tiradas en **11 segundos** (medido el 2026-09-25). Mercado a mercado, 500 estrategias × 10 mercados (12 M de operaciones) en **113 s** y 7,3 GB (medido el 2026-09-26). No toca SQX, así que puedes lanzarlo con la GUI abierta.

Los monos se pueden repetir, pero hay que pedirlo. Cada estrategia sortea de su propia semilla, que sale de la semilla raíz de la corrida, de su nombre, del peldaño y del bloque. Sin `--seed` la raíz se sortea nueva cada vez (el dueño quiso monos distintos en cada corrida, 2026-09-29), así que una estrategia con p = 0,04 puede leer 0,06 a la siguiente. **Cuando el p del mono se usa como resultado de otro estudio — una calibración, una regla — pasa `--seed <número>`**: con la misma raíz los p son idénticos, uses 1 proceso o 96, y aunque cambien las demás estrategias del lote (medido el 2026-10-02 sobre 300 estrategias:

`nulls.csv` y `manifest.json` byte a byte iguales; solo cambian la hora y los segundos que tardó, dentro de `monkey.json`). Para repetir una corrida que no llevaba `--seed`, coge la semilla que dejó apuntada en su `manifest.json`. Subir `nulls.draws` añade monos y no cambia los primeros. Lo que sí los cambia es `nulls.chunk_trades`: no lo toques. No toca SQX, así que puedes lanzarlo con la GUI abierta.

Lo que imprime `studies.readings.monkey.one`, salida real (2026-09-22, `Strategy 1.10.80` de `MC_Trades`, datos borrados el 2026-09-25):

```
Strategy 1.10.80   399 operaciones   muestra OOS1   2,500 monos por peldano
reconciliacion 0.999985  (relleno open-open, segunda convencion 0.9629)

           tu estrategia    mono medio     mono p95  percentil        p
-------------------------------------------------------------------------
     net       21,387.13     -5,567.68    19,567.73      96.2%   0.0380  <--
  sharpe            0.09         -0.02         0.07      98.6%   0.0148  <--
      pf            1.31          0.95         1.25      97.3%   0.0276  <--
   retdd            3.18          0.02         1.97      98.2%   0.0184  <--
      dd         6,724.18     20,180.50    37,685.82       1.4%   0.0144  <--

de donde sale la ventaja, medida en 'net':
  ventaja total sobre el mono suelto ....      26,960.91
  de cuanto aguanta la posicion .........         156.05
  del tamano por volatilidad ............         103.83
  resto, que es el momento de entrar ....      26,701.03
```

Se lee así: **el mono medio pierde 5.568 $ y tu estrategia gana 21.387**; solo el 3,8 % de las 2.500 tiradas al azar igualó eso. En drawdown está en el percentil 1,4 — sufrió mucho menos que el azar, que es lo contrario de lo que pasa con el beneficio y por eso su columna se lee al revés.

Y la atribución dice de dónde sale: de los 26.961 $ de ventaja, **26.701 son el momento de entrar**. La duración aporta 156 y el tamaño 104. En esta estrategia el edge es timing y poco más.

⚠ El «resto» no se mide aparte: es lo que queda tras quitar los canales que **sí** se pueden sortear. Si un día se añade un peldaño nuevo, ese resto se encoge.

Debajo va el histograma de las 2.500 tiradas con tu estrategia marcada, que es el dibujo del que va todo esto, y luego los avisos de `distrust`.

Qué produce

```
reports/<proyecto>/<databank>/<fecha>/nulls/
├── nulls.csv       una fila por estrategia
├── rungs.json      qué aleatoriza cada peldaño, para que el CSV se entienda solo
└── manifest.json   qué lo produjo, con qué export y cuántas tiradas
```

Los informes **se acumulan**: una corrida nueva de otro día no borra la de hoy.

Cómo se lee el resultado

La verificación primero. Salida real (misma estrategia y fecha):

```
Strategy 1.10.80  (399 trades, muestra OOS1)

RECONCILIACION  fill=open-open  corr=0.999985  gap mediano=6.39 $  (segunda mejor convencion:
0.9629)
BARRERAS        399 trades, 339 tocan una barrera sintetica, discrepancias vectorizado vs
bucle: 0
UNIFORMIDAD     400 runs nulos juzgados contra el resto: KS=0.0633 p=0.078 media=0.487 (debe
ser ~0.5)
```

Las tres tienen que pasar:

- **RECONCILIACION** por encima de 0.99. Es la que lo sostiene todo: si el P/L que reconstruimos desde las velas no es el que SQX reportó, tampoco lo será el de las tiradas imaginarias. Aquí 0.999985. La "segunda mejor convención" en 0.9629 dice que el relleno está identificado sin ambigüedad: SQX rellena **a la apertura de la vela**, en la entrada y en la salida.
- **BARRERAS** tiene que dar **0 discrepancias**. Tus estrategias de hoy no llevan stop ni objetivo, así que el código que los calcula no lo ejercita ninguna operación real: se le ponen barreras inventadas a propósito y se compara el cálculo rápido contra un bucle lento.
- **UNIFORMIDAD** con media ~0.5. Se coge una tirada al azar, se hace pasar por la estrategia real y se mide su percentil contra las demás. Si el mecanismo es correcto, esos percentiles salen repartidos de 0 a 1. Si salieran torcidos, habría un fallo en el sorteo — y se ve **antes** de mirar ninguna estrategia de verdad.

Y luego el CSV. Las columnas:

columna	qué es
`n`	operaciones en la muestra
`reconcile`	la correlación de arriba, por estrategia
`real_net`, `real_sharpe`, …	lo que hizo la estrategia de verdad
`p_timing_net`, `p_timing_sharpe`, …	**el percentil**: qué parte de las 2.500 tiradas la igualó o la superó
`p_free_net`, …	lo mismo con el mono más suelto
`edge_total`	cuánto le sacó al mono, en dólares
`edge_sizing`, `edge_holding_time`	cuánto de eso venía del tamaño y de la duración
`distrust`	por qué no fiarte, en cristiano

Un `p` por debajo de 0.05 quiere decir que menos de 1 de cada 20 tiradas al azar lo igualó.

Un ejemplo completo

Corrida real del 2026-09-22 sobre las 757 estrategias de `MC_Trades` (datos borrados el 2026-09-25; los comandos de arriba corren hoy sobre las 36 de `Results`), peldaño `timing` (solo se aleatoriza *cuándo* entra):

```
estrategias: 757   reconcile mediana: 0.999983   minimo: 0.99945

pasan a p<0.05, por estadistico:
  dd       640 de 757 =  84.5%    p mediano 0.013
  sharpe   586 de 757 =  77.4%    p mediano 0.027
  retdd    582 de 757 =  76.9%    p mediano 0.028
  pf       418 de 757 =  55.2%    p mediano 0.046
  net      299 de 757 =  39.5%    p mediano 0.059

la media del mono es NEGATIVA en el 100% de las estrategias (mediana -4.698 $)
```

Esto es lo más importante de toda la página, y hay que leerlo despacio:

Es la **misma** simulación. Lo único que cambia es qué número se compara. Y pasa del 84.5 % al 39.5 %. **La elección del estadístico decide el veredicto mucho más que la elección del mono.**

Por qué: el Sharpe divide el beneficio entre la volatilidad, así que borra justo lo que separa a tus operaciones de las aleatorias — las tuyas son un **36 % menos volátiles** (491 $ contra 661 $ de desviación por operación). Ser más tranquilo que el azar **es** un edge, y el Sharpe te lo paga mientras que el beneficio neto no.

Por eso el informe imprime los cinco y **no elige ninguno**. Elegir es tuyo.

Y la segunda lectura: la media del mono es negativa en las 757. Un mono que entra al azar en el oro **pierde dinero**, porque con posiciones de 6 horas captura ~2.867 $ de la subida del oro y paga ~7.756 $ de coste. Batir a ese mono es, por tanto, un listón **más bajo** que ganar dinero.

El tercer comando: ¿y la población entera?

```
python3 -m studies.screening.monkeyExcess.report --project XAUUSD --databank Results
```

Los dos comandos de arriba miran estrategias. Éste mira **el lote**, y contesta otra pregunta: *elegir también puede salir con suerte.* Si coges 757 estrategias sin ningún edge y te quedas con las que sacan `p<0.05`, te llevas 38 igualmente — porque `p<0.05` significa "esto le pasa a 1 de cada 20 por azar", y 757/20 = 38.

Salida real:

```
| estadistico | pasan | si NINGUNA tuviera edge | exceso | de las pasadas, suerte | nombrables
|
|---|---|---|---|---|---|
| `net`    | 299 | 38 | **+261** | 13% |   0 |
| `sharpe` | 586 | 38 | **+548** |  6% | 152 |
| `pf`     | 418 | 38 | **+380** |  9% |   0 |
| `retdd`  | 582 | 38 | **+544** |  7% | 243 |
| `dd`     | 640 | 38 | **+602** |  6% | 614 |
```

Dos lecturas, y son independientes:

- **exceso** — cuántas hay de verdad. 299 pasan y el azar daría 38: sobran 261. **Hay señal.** Éste es el número que juzga a tu generador, y es comparable entre tiradas: si cambias el template y el exceso baja, lo has empeorado.
- **nombrables** — a cuántas puedes señalar con el dedo. En `net`, **ninguna**. No es que no haya señal, es que ninguna destaca lo bastante entre 757 candidatas.

Esa discrepancia no es un fallo: *"aquí dentro hay unas 261 buenas y no te puedo decir cuáles"* es una conclusión legítima y frecuente.

⚠ **El aviso de muestra preseleccionada.** Si más del 95 % de las estrategias gana dinero en la muestra que se está juzgando, el informe lo canta arriba del todo: esa databank fue filtrada **por** esa muestra, y entonces el exceso mide el filtro y no el generador. En `MC Trades` salta, porque desciende de una tarea con condiciones de aceptación sobre OOS. Para juzgar al generador hace falta una databank sin esas condiciones.

Qué NO te dice

- **No te dice que la estrategia vaya a funcionar.** Dice que su momento de entrar, en esta historia concreta, fue mejor que entrar al azar en esa misma historia.
- **No te dice que el edge sea significativo.** Batir al mono es más fácil que batir a cero, porque el mono pierde por costes. Para "¿puede su media ser cero?" está MinTRL en `core/significance.py` — y son tests **distintos**: con el beneficio neto, 48 estrategias pasan MinTRL y fallan el mono, y 145 al revés.
- **No corrige por haber buscado.** Estas estrategias salieron de generar miles y quedarse con las buenas. Si vas a quedarte con las mejores de una lista de 200, el `p` de cada una hay que corregirlo (Benjamini-Hochberg), y el `p` más pequeño que se puede leer con 2.500 tiradas es `1/2501 = 4.0e-4`. Si te salen `p` topados ahí, el `distrust` te lo dice y hay que subir `nulls.draws`.
- **No vale para el tramo `oos2`.**
- **Los costes de XAUUSD son provisionales** (los que trae SQX de fábrica, no pactados con Infinox). Como la posición del mono la fija sobre todo el coste, estos `p` se mueven con ellos. El `distrust` lo repite en cada fila a propósito.

Si algo falla

- **`un trade dura N barras y barrier.max_hold es 400`** — hay una operación más larga que el techo del escaneo. Sube `barrier.max_hold`; el módulo se niega en vez de recortar la operación, porque recortarla sería valorar otra operación distinta de la que SQX corrió.
- **`RECONCILIACION` por debajo de 0.99 en el `distrust`** — el `--timeframe` no es el de la estrategia, o el `--feed` no es el mercado en el que corrió. Es el error más común.
- **CSV vacío** — ninguna estrategia llega a `verdict.min_trades` (30) en esa muestra. Revisa que `--sample` sea el correcto: en este corpus es `OOS1`, no `OOS`.

50. Edge sobre el coste pagado

Qué pregunta responde

De cada operación que hizo una estrategia **en oos1**, ¿cuánto ganó antes de pagar costes, comparado con lo que pagó? Cada operación se repercia desde la apertura de las barras con el repreciado común (el mismo que usa la puerta del paso 8) y los costes declarados de la tarea: spread, doble slippage, comisión y swap por noche. El resultado es un solo número por estrategia:

`edge = media(P/L + coste) / media(coste)`

Vale 1 en el punto muerto: la estrategia gana justo lo que paga. Es también el múltiplo del coste al que su ventaja neta llegaría a cero. Es **el mismo número** que `edge_mean_oos` de la puerta sobre las mismas operaciones (lo ata `tests/test_edgecost.py`).

Cambio 2026-10-07 (encargo 44): ya no se reconstruye el bruto con los precios de relleno de SQX, no se mide el spread contra las barras, y no hay mediana, número de operaciones ni desglose por coste (comisión, swap, spread, slippage por separado): el repreciado devuelve un coste, no sus partes. Tampoco entra el build: sólo oos1.

Cuándo lo usas

En dos sitios del WORKFLOW: el paso 8, como lectura sobre lo que la puerta deja, y el paso 25, al final, sobre la versión que se va a operar. **No decide nada por ti**: `min_edge_spreads` y `action` son del dueño; el módulo mide y, con `action: drop`, escribe el veredicto que `curate` puede aplicar.

Antes de empezar

- Una cosecha ya hecha (`studies.screening.gate.harvest`) o el export de la versión final, con la misma forma de columnas. El módulo no toca SQX.
- El feed tiene fichero en `assets/symbols/` y sus barras en `~/Desktop/AlgoData/bars/<feed>/`.
- El repreciado rechaza las fichas de forex (swap en puntos, pendiente del dueño).

Cómo se ejecuta

```
python3 -m studies.readings.edgeCost.report --project Trade_XAUUSD_M30_ejemplo --databank
Results --feed XAUUSD_M1
python3 -m studies.readings.edgeCost.report --project Trade_XAUUSD_M30_ejemplo --databank
Results --feed XAUUSD_M1 --strategy "Strategy 14.19.58"
```

flag	obligatorio	qué hace
`--project`, `--databank`	sí	de qué cosecha lee (el databank de build); coge siempre la más reciente
`--feed`	sí	feed de SQX con el que se repercia, p. ej. `XAUUSD_M1`
`--strategy`	no	una sola estrategia, leída en detalle; sin él, toda la población
`--set`	no	cambia un umbral sin editar el fichero: `--set verdict.min_edge_spreads=3`

Qué produce

`~/Desktop/AlgoData/reports/<proyecto>/<databank>/<fecha>/edgeCost/`:

fichero	qué es
`edgeCost.json` / `.html` / `.md`	el panel de la población: bruto medio, coste medio y edge por estrategia
`verdict.csv`	`identity`, `edge_mean`, `verdict` (MANTENER/DESCARTAR)
`estrategias/<nombre>.json` / `.html`	una estrategia: sus números y el bruto por hora y día de entrada
`manifest.json`	de qué cosecha salió

Cómo se lee

Verde: edge por encima del umbral (2, del dueño); rojo: por debajo. Con `action: mark` sólo colorea; con `drop`, `verdict.csv` pide DESCARTAR a quien no llega. El bruto por hora y día es la huella de la sesión en el bruto, no una medida del spread por hora.

Qué NO te dice

- No juzga si el coste declarado es el real: mientras `assets/symbols/<SYMBOL>.yaml` lleve valores `PROVISIONAL`, el resultado lo avisa y hereda esa duda.
- Sin verificar contra SQX en el repreciado: cortos, comisión distinta de cero y swap en puntos.
- No usa el build ni oos2.

Si algo falla

- `ningún fichero de assets/symbols/ declara el feed …` — el `--feed` no tiene ficha.
- `una operación está sellada fuera de la rejilla de barras` — la operación cerró dentro de una barra (típico de una versión con stop del paso 25): el repreciado sólo entra y sale a aperturas de barra, y se niega en vez de aproximar. Pendiente de decisión del dueño.

42. La calidad de la entrada — ¿la señal vale algo por sí sola?

Qué pregunta responde

Una estrategia gana por tres cosas mezcladas: cuándo entra, cuándo sale, y que el mercado se moviera. Este comando aísla **la primera**, y lo hace sin volver a correr nada en SQX: coge cada entrada, camina las velas hacia delante y mide cuánto corrió el precio a favor y cuánto en contra, ignorando por completo las salidas de la estrategia.

Dos preguntas:

1. **¿Predice la entrada movimiento favorable?** Se compara el recorrido a favor contra el recorrido en contra (el *e-ratio*), y ese número se contrasta contra **entradas al azar a las mismas horas y con la misma proporción de largos**. Si tu curva cae dentro de la banda del azar, la entrada no aporta: lo que ganas viene de las salidas o de la deriva del mercado.
2. **¿Cuánto edge se pierde llegando tarde?** Si entrar una vela después se lleva media ventaja, o el sistema es frágil a la latencia, o hay información del futuro metida en la señal.

Cuándo lo usas, y cuándo no

Lo usas cuando quieres saber qué parte de la estrategia es la señal. Es el complemento natural del test del mono (`06-lecturas.pdf › 26-nulos`): aquél pregunta si el resultado bate al azar, éste **dónde vive** la diferencia.

No lo uses para descartar sin más. Una estrategia puede salir `no_signal` y ser rentable: lo que te está diciendo es que su ventaja está en la salida o en la deriva, y eso es un objeto distinto que hay que reconocer, no necesariamente uno malo.

No sirve si la estrategia lleva stop o target para la parte del retraso. Ahí las salidas se mueven con la entrada y el tier 1 deja de valer; hace falta el simulador de replay, que todavía no existe (`docs/encargos/16-replay-de-operaciones.md`). La población XAUUSD actual no lleva ninguno, por eso aquí sí se puede leer.

Antes de empezar

Dos cosas:

- un `trades.parquet` exportado (`python3 -m sqx.export.export_trades …`);
- las barras del feed en la librería (`03-datos-costes-y-registro.pdf › 13-barras`). El comando lee el M1 y el timeframe de la estrategia; si el timeframe no estaba cacheado, la primera corrida tarda medio segundo más.

Cómo se ejecuta

```
python3 -m studies.readings.entryQuality.report \
    --export ~/Desktop/AlgoData/raw/XAUUSD/Results/2026-09-03/trades.parquet \
    --strategy "Strategy 1.17.44"
```

flag	obligatorio	qué hace
`--export`	sí	el `trades.parquet`
`--strategy`	sí	el nombre exacto con el que aparece en el export
`--set`	no	`--set run.timeframe=H1`, `--set run.feed=USDJPY_DukasM1_the5ers`, `--set eratio.draws=500`

⚠ **`run.feed` y `run.timeframe` del `config.yaml` son los del último activo estudiado**, no una propiedad del export. Si analizas otro símbolo hay que pasarlos, o las velas serán las del oro y no lo dirá nadie.

3,3 segundos con 1.119 operaciones y 200 sorteos. No toca SQX.

Qué produce

Sólo salida por pantalla, dos bloques.

Cómo se lee el resultado

Bloque 3 — el e-ratio. Para cada horizonte `k` (en velas desde la entrada):

columna	qué es
`e(k)`	recorrido a favor ÷ recorrido en contra, los dos en unidades de la volatilidad de esa entrada
`p5`, `mediana`, `p95`	lo mismo para entradas **al azar**, a las mismas horas y con los mismos largos/cortos
`sobre la banda`	si tu entrada supera el percentil 95 del azar

`e(k) ≈ 1` o dentro de la banda = la entrada no aporta. Y el `k` donde `e(k)` hace máximo es el **horizonte natural de la señal**: compáralo con la duración mediana real de las operaciones. Si tu señal deja de valer a las 13 velas y tú aguantas 30, estás devolviendo lo ganado.

Debajo se imprime el e-ratio **por dirección**. Una estrategia cuyos largos lo sostienen todo es otra cosa distinta de una cuyas dos mitades funcionan, y la curva agregada no enseña ni una ni otra.

Bloque 4 — el retraso. Por cada `d` (velas, y también minutos):

columna	qué es
`entregado`	cuánto precio regalas por entrar tarde, en dólares de cuenta
`esperanza_neta`	lo que quedaría de tu esperanza por operación
`DCR`	lo entregado como fracción de la esperanza **bruta**
`veces_el_coste`	lo entregado como múltiplo de lo que el backtest ya te cobraba

`DCR` por encima de 0,3 con **una sola vela** de retraso es la señal de alarma. Negativo significa que esperar te habría salido a favor, que es lo contrario de la fragilidad.

⚠ El PDF original pide esto en **unidades de spread**. En este install el spread va dentro de los precios de fill y no aparece en el residual `bruto − neto` (`knowhow/costs/where-the-spread-is.md`), así que lo recuperable por operación es el coste total, no el spread solo. La columna se llama `veces_el_coste` por eso, y leerla como spreads exageraría la gravedad.

Un ejemplo completo

Salida real sobre `Strategy 35.44.31` de `MC_Trades` (datos borrados el 2026-09-25; el comando de arriba corre hoy sobre `Strategy 1.17.44` de `Results`):

```
Strategy 35.44.31 · muestra OOS1 · XAUUSD_DukasM1_Infinox M30 · 1119 de 1119 operaciones con
camino completo

-- 3 · calidad de la entrada (MFE/MAE contra entradas al azar)
     e(k)     p5  mediana    p95  sobre la banda
k
1   0.894  0.898    1.038  1.221           False
5   1.013  0.904    1.005  1.102           False
10  1.067  0.917    1.005  1.099           False
20  1.075  0.924    1.004  1.090           False
50  1.059  0.945    1.021  1.110           False
largos (1119): k=1 0.89 · k=5 1.01 · k=10 1.07 · k=20 1.08 · k=50 1.06
máximo de e(k) en k=13 · duración mediana real 0 days 07:30:00
-> no_signal: la entrada no se distingue de entrar al azar a las mismas horas

-- 4 · lo que cuesta llegar tarde (tier 1, salidas sin mover)
en barras de M30:
   entregado  esperanza_neta    DCR  veces_el_coste
d
1     -1.582          18.227 -0.043          -0.080
2      1.433          15.212  0.039           0.073
3      1.382          15.262  0.038           0.070
4      5.694          10.950  0.157           0.289
5     17.950          -1.305  0.493           0.910
en minutos:
   entregado  esperanza_neta    DCR  veces_el_coste
d
1     -1.939          18.583 -0.053          -0.098
2     -0.994          17.639 -0.027          -0.050
3     -0.000          16.645 -0.000          -0.000
4     -1.533          18.177 -0.042          -0.078
5     -2.647          19.291 -0.073          -0.134
-> latency_robust: el edge sobrevive a entrar tarde
```

Qué dice esto de verdad:

- **La entrada no aporta nada.** En los cinco horizontes la curva real está dentro de la banda del azar, y en `k=1` incluso por debajo. Lo que gana esta estrategia no lo gana por acertar cuándo entra.
- **Es enteramente larga**: 1.119 operaciones, cero cortas. En un activo que subió en el periodo, eso es exactamente lo que la banda ya tiene en cuenta — por eso la comparación se hace con los lados barajados y no redibujados.

- **El horizonte de la señal son 13 velas** (6,5 horas) y la duración mediana real es 7,5 horas. Coinciden, así que al menos la salida no está muy desalineada con lo poco que la entrada dice.
- **La latencia no la mata**: hasta tres velas de retraso es indiferente, y al minuto es incluso favorable. Combinado con lo anterior tiene sentido: no se puede perder por llegar tarde a una señal que no marcaba nada.

Qué NO te dice

- **Nada de las salidas**, a propósito. Un `no_signal` rentable significa que la ventaja está ahí, y eso se mide en otro sitio.
- **No distingue fragilidad de latencia de información del futuro.** Un DCR alto es una de las dos y este test no puede separarlas.
- **Con stop o target, el bloque 4 no vale.** Las salidas se moverían con la entrada.

Si algo falla

- **Menos operaciones con «camino completo» que en total**: las que empiezan cerca del final de los datos se descartan a propósito, porque su camino se cortaría a un horizonte distinto del resto.
- Si `e(k)` sale absurda (cientos), casi seguro es que el `--set run.feed` no es el del export y estás midiendo las entradas de un activo sobre las velas de otro.

53. El mapa condicional — ¿de qué depende que esto siga funcionando?

> ⚠ **Este mapa fabrica hipótesis, no las comprueba.** Corta la muestra en volatilidad x tendencia y en día de la semana; con tres cortes y varias celdas por cada uno, alguna sale significativa por azar en **cualquier** estrategia, incluida una sin ninguna ventaja real. Es el único test del PDF del dueño (`TRADE_LEVEL_TESTS.pdf`, item 6) que funciona así, y por eso va el último de la tanda y con condiciones: **ninguna celda de aquí es un filtro**. Si una celda parece decir algo, se anota como candidata en `ledger/thresholds.yaml` (encargo 8) y se revalida sobre datos que todavía no se hayan mirado — nunca se aplica directamente sobre lo que este mapa ya vio.

Qué pregunta responde

No «¿qué filtro añado?», sino **«¿de qué depende que esto siga funcionando?»**. Clasifica cada operación por el estado del mercado **en el momento de entrar** — sólo con información disponible entonces — y enseña, celda a celda, cuántas operaciones hay, cuánto ganan de media (con su intervalo) y qué parte acierta. Una estrategia que sólo gana en volatilidad alta depende de que ese régimen vuelva: es una propiedad del objeto, no un defecto a corregir.

Cuándo lo usas, y cuándo no

Lo usas para escribir en la ficha de una estrategia de qué depende su edge — al lado de la concentración temporal de los beneficios (la misma idea sobre el eje del calendario). Es lectura, no criba.

No lo uses para decidir un filtro sin pasar por el ledger.

Antes de empezar

- Una cosecha de la puerta OOS ya hecha (`studies/screening/gate/harvest.py`): el mapa lee su `trades.parquet` (operaciones por `identity`) y su `metrics.parquet` (el nombre de cada estrategia), nunca un export directo de SQX ni una instalación viva.
- Las barras del feed en la librería (`03-datos-costes-y-registro.pdf › 13-barras`): el mapa resuelve un candil diario a partir de la rejilla de la estrategia.
- El tramo `build` del activo decidido en `assets/_policy.yaml` — de ahí salen los cortes de tercil, congelados. Sin tramo `build`, el comando se detiene con el mismo error que `core.assets` (regla dura 5).

Cómo se ejecuta

```
python3 -m studies.readings.conditionalMap.report \
    --harvest ~/Desktop/AlgoData/harvest/XAU_ISOOS_ejemplo/Results/2026-09-23 \
    --strategy "Strategy 5.16.81" \
    --set run.symbol=XAUUSD run.feed=XAUUSD_DukasM1_Infinox run.timeframe=M30
```

flag	obligatorio	qué hace
`--harvest`	sí	la carpeta del día de una cosecha (`.../harvest/<proyecto>/<databank>/<día>`)
`--strategy`	sí	el nombre exacto con el que la cosecha lo tiene en `metrics.parquet`
`--set`	no	`--set run.symbol=USDJPY run.feed=USDJPY_DukasM1_the5ers run.timeframe=H1` , `--set volatility.atr_period=30` , `--set trend.window=10`

⚠ **`run.symbol`, `run.feed` y `run.timeframe` de `config.yaml` son los del último activo estudiado**, no una propiedad de la cosecha: analizar otro símbolo sin pasarlos clasifica las operaciones contra las velas y el tramo de construcción equivocados, sin que nada avise.

Un segundo con 361 operaciones y 9.999 remuestreos por celda poblada. No toca SQX.

La muestra: OOS1 o Completa

Las dos pestañas llevan arriba el desplegable **«Muestra»** (2026-10-01):

- **OOS1** — sólo las operaciones del primer fuera de muestra. Es con la que abre (`run.sample`).
- **Completa** — todo lo que la cosecha tiene de la estrategia: dentro de muestra (`build`) más OOS1, el backtest entero que se cosechó. `oos2` nunca entra en una cosecha.

El estudio calcula las dos en la misma pasada, así que el desplegable cambia sin volver a correr nada. La razón es aritmética: partir 300-500 operaciones de OOS1 en 30 casillas sesión x día deja casi todas por debajo de 30. En las 15 madres de `Test_USDJPY_donchianUpperCrossUp_H1` , OOS1 llena entre 0 y 4 de las 30 casillas y la muestra Completa entre 10 y 23 (`Strategy 10.1.79` : 513 operaciones en OOS1 y 4 casillas; 1.550 en Completa y 23).

⚠ **La Completa incluye el tramo en el que se construyó la estrategia.** Lo que se vea ahí puede ser parte de lo que la construcción ajustó; una celda que sólo destaca en la Completa y no en OOS1 es, todavía más, una hipótesis y no un hallazgo. Los cortes de tercil son los mismos en las dos.

Qué produce

Sólo salida por pantalla (Markdown) y, además, el resultado de la estrategia bajo `reports/<proyecto>/<databank>/<día>/conditionalMap/estrategias/<estrategia>.json` y `.html` — lo que pinta la ventana.

Cómo se lee el resultado

Pestaña «Volatilidad x tendencia en la entrada». Una rejilla 3x3: filas el tercil de volatilidad realizada (ATR diario) **del día anterior a la entrada**, columnas el tercil de tendencia (ratio de eficiencia de Kaufman) **del mismo día anterior** — nunca el día de la propia entrada, porque ese día incluye barras que todavía no habían pasado en el momento de entrar. Los dos tipos de corte salen **una sola vez**, sobre el tramo `build` del activo, y se aplican sin cambios a la muestra que se lee. Una celda que dice **«< 30 ops»** no es cero: es que tiene menos operaciones que el suelo de la puerta OOS

(`engines/nulls/config.yaml#verdict.min_trades` , hoy 30) y no enseña su P&L ni se juzga — pero lo dice, y la nota bajo el mapa cuenta cuántas son (desde el 2026-10-01; antes quedaban en blanco). La tabla de abajo trae, celda a celda, operaciones, P&L medio, su intervalo por bootstrap y el acierto.

⚠ **Corregido 2026-09-26** (revisión de esta misma tanda): la primera versión clasificaba cada operación con el candil diario de **su propio** día de entrada, que `engines.regimes.regime.daily()` construye con la sesión entera — en una operación real verificada, el 79 % de las barras de ese día llegaban **después** de la entrada, incluida la que ponía el máximo del día. Los cortes de tercil congelados sobre `build` heredaban el mismo defecto. `studies/readings/conditionalMap/regime.py` ahora desplaza la volatilidad y la tendencia un día completo antes de clasificar nada — una entrada del día D lee siempre lo que se sabía al cierre de D-1 —, y `tests/test_conditionalmap.py` prueba que una barra que llega después de la entrada, en el propio día de la entrada, no puede mover su celda. Los números de los dos ejemplos de abajo son los de la versión corregida y no coinciden con los de una ejecución anterior a esa fecha.

Pestaña «Sesión y día de la semana». Lo mismo por sesión de mercado y por día, con el mismo suelo, en tres vistas: sesión x día, sólo sesiones y sólo días (la ventana cambia entre ellas). Una sesión o un día por debajo de 30 operaciones no tiene barra, pero la nota bajo el gráfico lo nombra con sus operaciones («Sin mostrar por tener menos de 30 operaciones: Fuera de sesión (14)»). Las sesiones son las horas de trabajo de cada ciudad **en su propia hora local** —Tokio 9–18, Londres 8–17, Nueva York 8–17—, así que el cambio de horario de cada una mueve sus bordes en su fecha: Asia, solape Asia-Londres, Londres, solape Londres-NY, Nueva York y fuera de sesión.

⚠ **Las horas de SQX no son UTC.** Cada feed viene en la hora de su broker: los de Infinox en `EET` , los de the5ers en hora de Israel, el Brent en `EETUS` (la de Nueva York más 7 horas). El mapa lee la zona de cada feed del registro de SQX y pasa cada entrada a UTC antes de asignarle sesión; sin eso, todas caerían dos o tres horas desplazadas. La hora que el cambio de horario repite o salta no se asigna a ninguna sesión, y la nota de la pestaña dice cuántas son. El día de la semana es el del reloj del feed.

Ejemplo real, `XAU_ISOOS_ejemplo / Strategy 5.16.81` (361 operaciones OOS1, tramo build 2008–2017; las 331 de la tabla son las que caen en las seis celdas con población — el resto se reparte en las tres celdas de volatilidad media, ninguna de las cuales llega a 30):

```
Terciles congelados sobre el tramo de construcción de XAUUSD. ATR(20) para la volatilidad —
cortes [16.6062, 20.7802]. Ratio de eficiencia a 20 días para la tendencia — cortes
[0.1411, 0.3049].

| volatilidad | tendencia | operaciones | pnl_medio | ci_baja | ci_alta | acierto |
|---|---|---|---|---|---|---|
| baja  | baja  | 46 | -244.22 | -584.40 |  104.63 | 0.391 |
| baja  | media | 47 |  -55.25 | -398.26 |  269.87 | 0.553 |
| baja  | alta  | 47 |  123.11 | -194.50 |  446.41 | 0.532 |
| alta  | baja  | 59 |  -94.38 | -457.18 |  247.35 | 0.508 |
| alta  | media | 63 |   84.21 | -174.38 |  337.98 | 0.540 |
| alta  | alta  | 69 |  -14.68 | -253.98 |  221.13 | 0.449 |
```

(La celda «media» de volatilidad no aparece: ninguna de sus tres combinaciones llegó a 30 operaciones en esta muestra — 361 operaciones repartidas en 9 celdas se quedan cortas casi siempre, y es la razón de que el mapa esté pensado para leerse, no para recortar.)

Léelo así: seis de las nueve celdas posibles tienen población, y **ninguna** tiene el intervalo del P&L medio enteramente por encima o por debajo de cero — ni siquiera `baja x baja` (-584.40 a 104.63), la más negativa de las seis. Eso es exactamente lo que el aviso de arriba predice: con seis celdas mirando y ninguna cruzando el listón con margen, no hay aquí ni un candidato razonable a anotar en el ledger, sólo una descripción sin nada que sostenga un filtro.

Un ejemplo completo

Con `USDJPY_emaCross_H1 / Strategy 14.19.58` (256 operaciones OOS1, todas con día y tercil válidos), sólo la fila `baja` de volatilidad tiene población — las otras dos terciles no llegan a 30 operaciones en ninguna de sus tres combinaciones de tendencia; las 207 de la tabla son las de esa fila:

volatilidad	tendencia	operaciones	pnl_medio	ci_baja	ci_alta	acierto
baja	baja	81	-47.64	-149.45	53.11	0.469
baja	media	67	-61.37	-171.36	48.51	0.507
baja	alta	59	1.48	-122.79	126.54	0.508

Por sesión, `USDJPY_emaCross_H1 / Strategy 14.19.58` (256 operaciones OOS1; ninguna casilla sesión x día llega a 30, así que en OOS1 el cruce no se enseña y un aviso lo dice — en la muestra Completa se llenan):

sesión	operaciones	pnl_medio	acierto
Asia	90	33.33	0.567
Londres	42	-99.72	0.452
Solape Londres-NY	54	-13.95	0.537
Nueva York	42	-21.75	0.452

Y por día de la semana:

día	operaciones	pnl_medio	acierto
lunes	50	47.23	0.580
martes	48	-19.69	0.521
miércoles	67	-73.36	0.478
jueves	45	8.17	0.556
viernes	46	-124.40	0.391

Los tres intervalos de volatilidad cruzan cero: nada aquí sostiene que la estrategia dependa de un régimen de volatilidad concreto. El viernes es la celda más llamativa (-124.40, acierto 0.391 en 46 operaciones) — con cinco días mirando, es exactamente el tipo de celda que el aviso de arriba avisa que va a aparecer por azar,

no una que se pueda usar para excluir el viernes sin más.

Qué NO te dice

- **No es un filtro, y ninguna celda lo es** por sí sola: es una descripción con comparaciones múltiples sin corregir. Aplicar una sin pasar por el ledger y sin revalidar sobre datos nuevos es la forma más barata de sobreajustar la cadena entera.
- **Las sesiones son una convención, no un dato del mercado.** Las horas de cada ciudad las fijó el dueño el 2026-09-26; con otras horas, otras celdas. Se cambian en `config.yaml`, bloque `sessions`.
- **Nada sobre las salidas.** Sólo mira el estado del mercado al entrar; el resto de la estrategia (salida, tamaño, gestión) no está aquí.
- **No corrige por comparaciones múltiples entre celdas.** El aviso lo dice, no lo esconde: leer seis o nueve celdas y coger la más extrema es exactamente el error que el aviso señala.

Si algo falla

- **`el tramo build de <SÍMBOLO> no tiene fechas decididas`** — `assets/_policy.yaml` no trae ese tramo para el activo; hay que decidirlo ahí antes de correr el mapa (regla dura 5).
- **`N operaciones abren en días que el fichero de barras no tiene`** (desde `engines/regimes/regime.tag`, si se usara directamente) — símbolo o feed equivocado en `--set run.feed=…`; el mapa usa su propia clasificación y no llama a esa función, pero el mismo síntoma (una malla de barras que no cubre las fechas de la cosecha) aparece como celdas «< 30 ops» en vez de un error: revisa `run.feed`.
- **Todas las celdas dicen «< 30 ops», o sale «el mapa no se enseña»** — o la muestra es pequeña (prueba la Completa en el desplegable «Muestra»), o `run.feed` / `run.timeframe` no son los del activo que se está leyendo (ver el aviso de `--set` más arriba).

51. Estructura — qué condición sostiene el filo, y si vive en la dirección de la entrada

Qué pregunta responde

Coge una estrategia superviviente y le quita **una condición de entrada cada vez** (ablación), y aparte le da **la vuelta a la orden** dejando las entradas donde están (inversión). Te dice qué condición aporta de verdad por operación, cuál no hace nada y si lo que gana la estrategia depende de hacia dónde entra. Es el paso 23 del workflow, después del mapa condicional (22) y antes del stop ATR (24).

Cuándo lo usas, y cuándo no

Con una estrategia que ya pasó los pasos 1–21, para entender de qué está hecho su resultado.

No sirve para elegir. Una ablación que sale mejor que la madre no es una estrategia mejor que adoptar: es una condición que no aportaba. Quitarla es decisión tuya, se anota en el ledger y se revalida aparte (encargo 12 §5). Tampoco sirve con estrategias que ya llevan stop, target o trailing: la fábrica se niega a invertirlas, porque la vuelta sólo es un espejo sin ellos.

Antes de empezar

1. `python3 -m core.assets USDJPY` en verde (regla dura 5). Hoy los costes de USDJPY son **provisionales**: spread 0.1, slippage 0.05, comisión 0, swap largo 0 y corto −10.4 puntos/noche.
2. El `.sqx` de la madre **copiado fuera de SQX**, a `AlgoData/structural/<proyecto>/mothers/`. El nombre del fichero es el de la estrategia. Da igual de qué databank la copies (`Results` u `OOS`): el control de identidad compara cada tramo contra lo que **ese mismo fichero** guarda para su propia ventana — el tramo `build` contra su IS guardado, `oos1` contra su OOS guardado, nunca uno contra el otro (arreglado el 2026-09-27, ver más abajo).
3. Un **proyecto custom** en el custodio con las tres patas WFC (regla dura 10):

```
bash python3 -m sqx.projects.builder Test_USDJPY_structural_v1 --purpose "variantes estructurales" \ --template ~/Desktop/AlgoData/templates/library/emaCloseAbove/template.sqx \ --symbol USDJPY --role custodian --timeframe H1 --workflow \ --session-from ~/Desktop/SQX_w2/user/projects/USDJPY_emaCross_H1/project.cfx python3 -m sqx.projects.wfc USDJPY --timeframe H1 \ --cfx ~/Desktop/SQX_w2/user/projects/USDJPY_structural_v1/project.cfx
```

4. **El custodio libre**: nadie corriendo, puerto 5070 caído, y una copia de `user/projects` antes de arrancar (regla dura 1). `execute` lo arranca y lo para él.

Cómo se ejecuta

Cinco comandos. Sólo el segundo toca SQX.

```
W=~/Desktop/AlgoData/structural/TestUSDJPY_Workflow_v1/batch1
P=USDJPY_structural_v1
python3 -m sqx.structural.make --mothers
~/Desktop/AlgoData/structural/TestUSDJPY_Workflow_v1/mothers --out $W
python3 -m sqx.variants.execute --work $W --project $P           # □ custodio, ~2 min
python3 -m sqx.structural.keep --work $W                         # ¿SQX corrió lo que se
fabricó?
python3 -m sqx.export.export_retest --project $P --role custodian --batch structure \
    --databank WFC_Build --databank WFC_OOS1                     # las dos patas, un solo
arranque de SQX
# --batch structure evita que el export del paso 24 (stopgrid), el mismo día y el mismo
# databank, se coma este trades.parquet (OPEN #74)
python3 -m studies.readings.structure.report --work $W --project $P \
    --databank WFC_Build WFC_OOS1 --feed USDJPY_DukasM1_the5ers --symbol USDJPY
```

`sqx.structural.make`:

flag	obligatorio	qué hace
`--mothers`	sí	carpeta con los `.sqx` de las madres; una tanda para todas
`--out`	sí	carpeta del lote en `AlgoData`. Se vacía `sqx/` al refabricar

`studies.readings.structure.report`:

flag	obligatorio	qué hace
`--work`	sí	el lote de `make`, ya corrido y con `keep` hecho
`--project`	sí	el proyecto custom donde se reteseó
`--databank`	sí	las patas que se leen. **`WFC_OOS2` se rechaza**: `oos2` está reservado al WFC y a la WFM, y esto es el paso 23
`--feed`	sí	el feed del mercado principal; los mercados adicionales se ignoran
`--symbol`	sí	el activo: da el point value y la comprobación del ledger
`--set`	no	`seccion.clave=valor` sobre `config.yaml`, p. ej. `--set verdict.alpha=0.01`

Fabricar y leer tardan un par de segundos. Lo que tarda es el custodio: cuatro ficheros por tres patas y diez mercados, unos dos minutos.

Qué produce

- `<lote>/sqx/S00O00.sqx` (la madre reconstruida), `S00A01…` (una ablación por condición), `S00I00.sqx` (la inversión) y `structure.parquet`: qué es cada fichero y qué contiene de verdad.
- `<lote>/retested/<pata>/` y `retained.parquet`: los ficheros que devolvió SQX y si conservan la edición.
- `AlgoData/raw/<proyecto>/<pata>/<fecha>/trades.parquet`: las operaciones de cada pata.
- `<lote>/estudios/structure/estrategias/<estrategia>.json` y `.html`, `verdict.csv` y `manifest.json`.

⚠ `execute` exporta también el panel de `oos2` (`retest_oos2.csv`). No lo abras: el estudio no lo lee.

Cómo se lee el resultado

Estructura — Strategy 23.1.53

Qué condición de entrada sostiene el filo, y si vive en la dirección de la entrada.
Diagnóstico, nunca selección.

1 · Qué aporta cada condición 2 · La misma entrada, al revés 3 · Controles

> **diagnostico** Diagnóstico, nunca selección (encargo 12 §5): una ablación que mejora no es una estrategia mejor. Quitar la condición es decisión del dueño, se anota en el ledger y se revalida aparte.

> **sin_stops** La inversión es espejo exacto sólo sin stop, target ni trailing; la fábrica se niega a invertir una estrategia que los lleve.

---

1 · Qué aporta cada condición

Una ablación por condición de entrada. Se lee por operación: quitar un filtro da más operaciones, y el beneficio total siempre favorece a la que opera más.

Aquí se ve la combinación por defecto de Tramo (build). Las demás están en el resultado JSON y en la ventana.

`tramo: build`

ΔM por condición

CONDICIÓN QUITADA	LECTURA	OPS MADRE	OPS SIN ELLA	ESPERANZA MADRE	ESPERANZA SIN ELLA	ΔE	P ΔE	SHARPE MADRE	SHARPE SIN ELLA
MABarClosesAbove	redundante	694	694	43.32	43.32	0	–	0.04838	0.04838
QQEValue1CrossAbove	no se distingue	694	1,980	43.32	7.32	36.00	0.09459	0.04838	0.008315

ΔE y ΔSharpe son madre menos ablación, por operación: positivo es lo que la condición añade. 'Entradas gemelas' es la parte de las entradas de la madre que la ablación también tomó; baja cuando la estrategia sólo lleva una posición.

`tramo: build`

ΔE por condición



`tramo: build`

MABarClosesAbove — redundante

Quitarla no cambia ni una operación: en esta historia la cumplen siempre las demás.
El fichero reteseado conserva la edición, así que no es que SQX la ignorase.

`tramo: build`

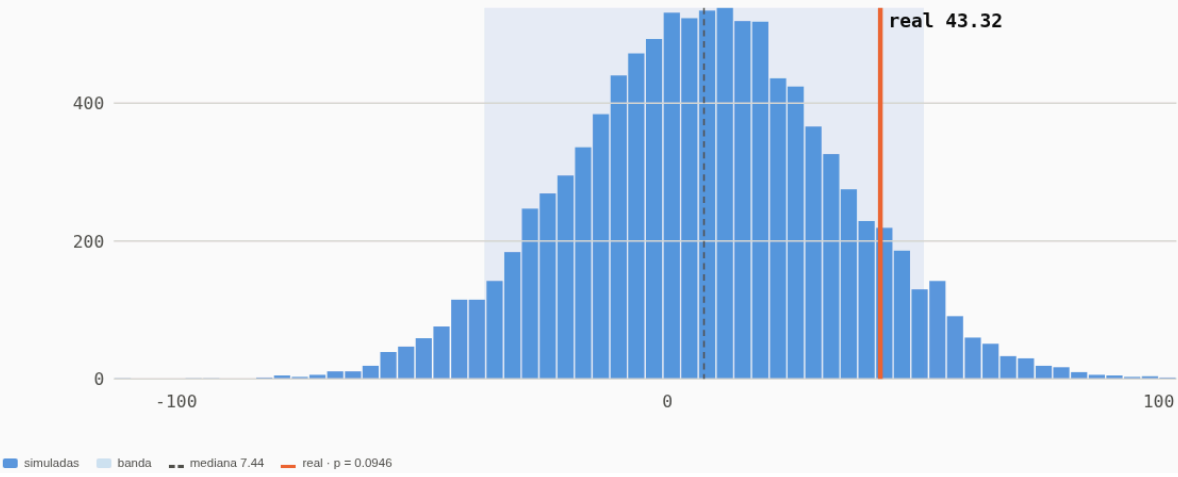
QQEValue1CrossAbove — no se distingue

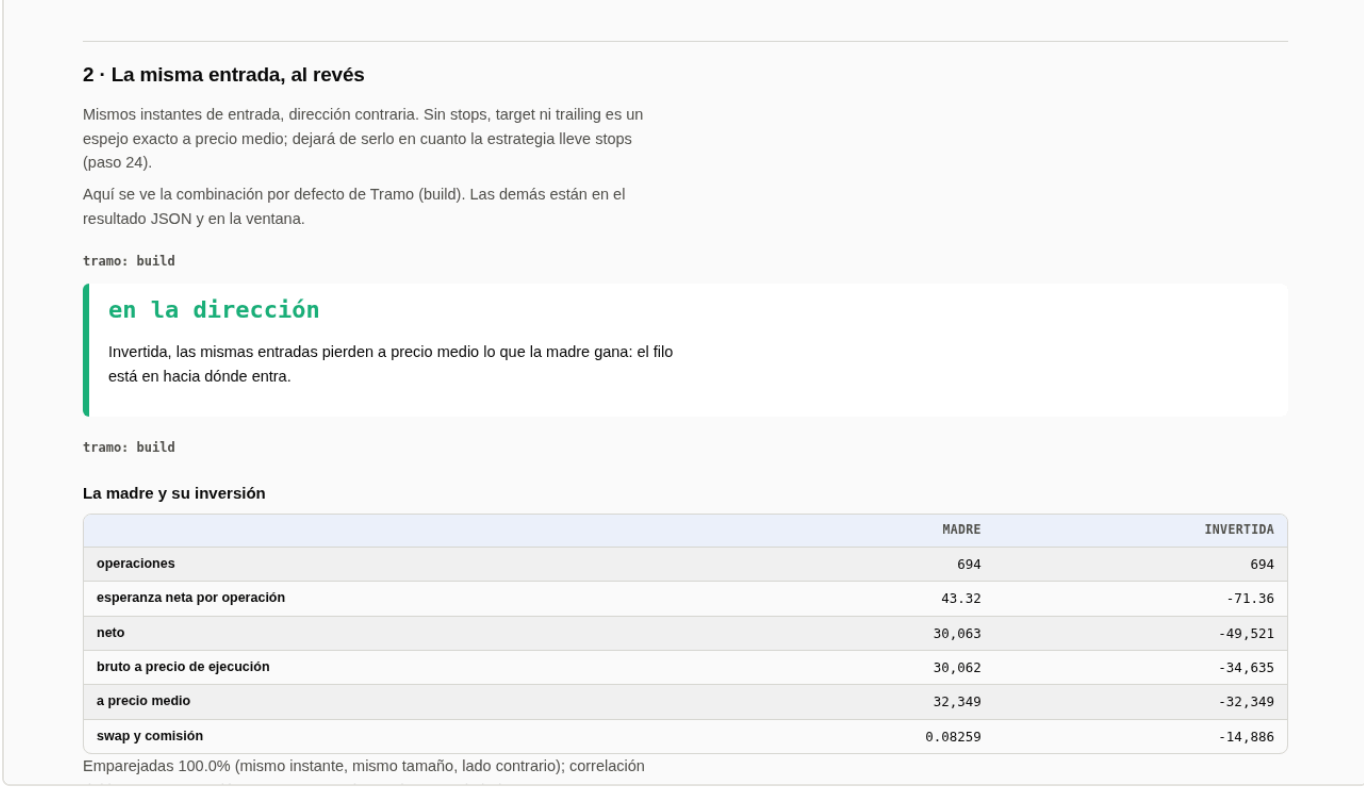
Con la condición la estrategia opera menos, pero lo que queda no es mejor que un recorte al azar del mismo tamaño: compra una muestra más pequeña y nada más.

`tramo: build`

Sin QQEValue1CrossAbove: 694 operaciones al azar

Esperanza por operación de recortes al azar de la estrategia sin la condición; la línea es la madre.





2 · La misma entrada, al revés

Mismos instantes de entrada, dirección contraria. Sin stops, target ni trailing es un espejo exacto a precio medio; dejará de serlo en cuanto la estrategia lleve stops (paso 24).

Aquí se ve la combinación por defecto de Tramo (build). Las demás están en el resultado JSON y en la ventana.

tramo: build

en la dirección

Invertida, las mismas entradas pierden a precio medio lo que la madre gana: el filo está en hacia dónde entra.

tramo: build

La madre y su inversión

	MADRE	INVERTIDA
operaciones	694	694
esperanza neta por operación	43.32	-71.36
neto	30,063	-49,521
bruto a precio de ejecución	30,062	-34,635
a precio medio	32,349	-32,349
swap y comisión	0.08259	-14,886

Emparejadas 100.0% (mismo instante, mismo tamaño, lado contrario); correlación

Pestaña 1, qué aporta cada condición. Todo va **por operación**: quitar un filtro da más operaciones y el beneficio total siempre favorece a la que opera más. ΔE es la esperanza de la madre menos la de la ablación: lo que la condición añade. `p ΔE` compara la madre con recortes al azar, del mismo tamaño, de la estrategia sin la condición. Las lecturas:

lectura	qué quiere decir
aporta	la condición elige operaciones mejores que un recorte al azar ($p < 0.05$)
no se distingue	opera menos, pero lo que queda no es mejor que recortar al azar
resta	sin ella la esperanza por operación es mayor. No se adopta: se anota y se revalida aparte
redundante	quitarla no cambia ni una operación, y SQX sí corrió el fichero sin ella
no es un filtro	sin ella opera igual o menos: era una rama de un OR, no un recorte
no aplicada	SQX devolvió un fichero distinto del fabricado: esa fila no se lee

Pestaña 2, la inversión. "A precio medio" es el movimiento sin spread, reconstruido de las dos operaciones emparejadas: la madre y su inversión son espejo exacto ahí. Lo que separa los netos es el spread (lo paga cada lado) y el swap. "En la dirección" quiere decir que el filo está en hacia dónde entra la estrategia.

Pestaña 3, controles. Sin los tres en verde no hay lectura: SQX conservó cada edición, alguna ablación cambia el número de operaciones, y la madre reconstruida reproduce un resultado que su fichero guardaba — **para la ventana de ese mismo tramo**, nunca la de otro. Hasta el 2026-09-26 el tramo `build` se comparaba contra el OOS que el fichero guardaba (bastaba con que los dos números coincidieran, vinieran de donde vinieran); arreglado el 2026-09-27: `build` compara contra el IS guardado, `oos1` contra el OOS guardado, cada uno el suyo.

Un ejemplo completo

`Strategy 23.1.53` de `TestUSDJPY_Workflow_v1`, USDJPY H1, 2026-09-26. Elegida por ser la de mejor OOS de las tres madres USDJPY (364 operaciones, PF 1.30) y la más simple: `EMA(50) close above` (la condición fija de la plantilla) AND `QQE(14) Value1 cruza por encima de 57.72`, salida a las 24 barras.

```
$ python3 -m sqx.structural.make --mothers .../mothers --out .../batch1
variant_id         strategy      kind               block direction   ok
    S00A01 Strategy 23.1.53  ablation    MABarClosesAbove         1 True
    S00A02 Strategy 23.1.53  ablation QQEValue1CrossAbove         1 True
    S00I00 Strategy 23.1.53 inversion                            -1 True
    S00O00 Strategy 23.1.53  identity                             1 True
1 madre(s) -> 4 ficheros en .../batch1/sqx

$ python3 -m sqx.variants.execute --work .../batch1 --project USDJPY_structural_v1
PROGRESS 100 4 variantes x 3 tramos, build: 4 en disco, oos1: 4 en disco, oos2: 4 en disco
```

Lo que devolvió SQX (panel de cada pata):

fichero	build: ops / neto	oos1: ops / neto
S00O00 madre reconstruida	694 / 30,062.63	364 / 35,062.48
S00A01 sin `MABarClosesAbove`	694 / 30,062.63	364 / 35,062.48
S00A02 sin `QQEValue1CrossAbove`	1,980 / 14,491.26	1,050 / 23,991.39
S00I00 invertida	694 / −49,520.95	364 / −49,031.31

La lectura (tramo oos1):

```
| condición quitada   | lectura     | ops madre | ops sin ella | esp. madre | esp. sin ella |
ΔE    | p ΔE  |
| MABarClosesAbove    | redundante  | 364       | 364          | 96.33      | 96.33         | 0
| –     |
| QQEValue1CrossAbove | aporta      | 364       | 1,050        | 96.33      | 22.85         |
73.48 | 0.031 |
inversión: en la dirección — emparejadas 100 %, a precio medio +36,609 / −36,609
controles: pass — la madre reconstruida = el OOS guardado (364 / 35,062.48)
```

Qué dice, en cristiano:

- **La condición fija de la plantilla no hace nada en esta estrategia.** Con el QQE cruzando 57.72, el precio ya está siempre por encima de la EMA(50): quitarla da las mismas 694 y 364 operaciones. No es que SQX ignorase la edición: el fichero reteseado ya no tiene el bloque.
- **El QQE es el que aporta**, pero sólo se distingue del azar en oos1 (p 0.031); en build, p 0.095.
- **El filo está en la dirección**: invertida, las mismas entradas pierden a precio medio exactamente lo que la madre gana. El neto invertido (−49,521) pierde más porque además paga spread y el swap de los cortos (−14,886 en build).

Qué NO te dice

- **No te dice qué estrategia usar.** Es diagnóstico. "Redundante" no es "bórrala": en otra historia la EMA podría filtrar algo.
- **No mide interacciones.** Quita las condiciones de una en una; dos que se tapan entre sí salen las dos como poco útiles.
- **No mira** `oos2`, a propósito.
- **Los costes son provisionales** (defaults de SQX, comisión cero): el ΔE compara dos estrategias con los mismos costes, así que se sostiene; el neto no.
- Sólo condiciones de **entrada**. Las salidas son otra prueba, y este corpus no tiene señales de salida.

Si algo falla

- `PermissionError: ledger: el paso 23 no puede mirar oos2` — has puesto `WFC_OOS2` en `--databank`. Quítalo.
- `el lote no es el plan` (make) o `SQX no conservó la edición` (keep) — un fichero no contiene lo que debía. No leas nada de ese lote: es el fallo de `OPEN.md` §9.
- `exit formulas [...]: inversion is only a mirror without stops` — la madre ya lleva stop o target. Este paso va antes del 24; con stops no hay inversión.
- No sale ninguna ablación — la señal de entrada tiene una sola condición: sin ella no quedaría regla de entrada, y la fábrica no la fabrica. Sólo salen la identidad y la inversión.